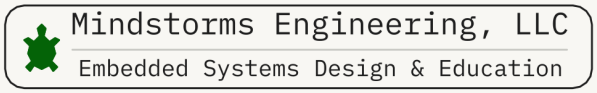
You Don't Need an RTOS (Part 2)

by Nathan Jones



At least, probably not as often as you think you do. Using a preemptive RTOS can help make a system schedulable (i.e. help all of its tasks meet their deadlines) but it comes at a cost: "concurrently" running tasks can interact in unforeseen ways that cause system failures, that dreaded class of errors known as "race conditions". In part, this is because humans have difficulty thinking about things that happen concurrently. Using the venerable superloop can help avoid this problem entirely and, in some cases, it can actually make a system work that wouldn't work at all using an RTOS! Additionally, a "superloop" style scheduler has the benefit of being simple enough for you to write yourself in a few dozen lines of code.

If you're unfamiliar with terms like "deadlines", "schedulability", or "worst-case response time", then I would argue you are almost *definitely* using an RTOS prematurely! If you *are* familiar with those terms, allow me to show you a few ways to improve the superloop that bring its performance nearly on par with an RTOS, with no concern about race conditions.

This is the second in a series of posts lauding the advantages of the simple "superloop" scheduler. Other articles in this series are or will be linked below as they are published.

- Part 1: Comparing Superloops and RTOSes
- Part 2: The "Superduperloop" (this article)
- Part 3: Other Cool Superloops and DIY IPC
- Part 4: More DIY IPC

Contents

Part 2: The "Superduperloop"

In this second article we'll tweak the simple superloop in three critical ways that will improve it's worst-case response time (WCRT) to be nearly as good as a preemptive RTOS ("real-time operating system"). We'll do this by adding **task priorities, interrupts, and finite state machines**. Additionally, we'll discuss how to incorporate a sleep mode when there's no work to be done and I'll also share with you a different variation on the superloop that can help schedule even the toughest of task sets.

A Quick Recap

In Part 1 of this series, we compared the superloop and RTOS schedulers. The simple superloop that we analyzed looked like this:

```
volatile int g_systick = 0;
void timerISR(void)
{
    g_systick++;
}
void main(void)
{
    while(1)
    {
        if( /* A is ready */ ) { /* Run Task A */ };
        if( /* B is ready */ ) { /* Run Task B */ };
        if( /* C is ready */ ) { /* Run Task C */ };
    }
}
```

The WCRT of any task ended up being equal to the sum of the WCETs of every task (equation 1 from Part 1), since the worst thing that could happen is for a task to become ready to run *juuuust*

after the scheduler evaluates its `if` expression as `false` and then *every* other task runs, taking their longest possible execution times when they do. In general, if each task's WCRT is less than its deadline then the system is schedulable; since every task has the same WCRT in this case, we only need to check if that's less than the shortest deadline of any task in the task set. If it is, then the system is schedulable.

Change #1: Adding Task Priorities using `if...else if`

Our first change is subtle, yet powerful: instead of coding our tasks as a list of separate `if`s, we're going to turn the whole thing into a *chained* `if` statement using `else if`s.

```
volatile int g_systick = 0;
void timerISR(void)
{
    g_systick++;
}
void main(void)
{
    while(1)
    {
        if     ( /* A is ready */ ) { /* Run Task A */ };
        else if( /* B is ready */ ) { /* Run Task B */ };
        else if( /* C is ready */ ) { /* Run Task C */ };
    }
}
```

What effect does this have? Since an `if` statement will only ever execute the first conditional path in the chain that evaluates to `true`, this code will find and execute the first ready-to-run task, jump to the bottom of the `if` statement, and then start back again at the top. If the tasks are sorted so that tasks with higher priority are situated higher up in the chain of `if` clauses (with the highest priority task being at the top of the `if` statement and the lowest priority task being at the bottom), then this change has the effect of *always running the highest priority task that's ready to run*. We've created task priorities just by adding some `else if`s!

Schedulability of the Superloop with Task Priorities (a.k.a. the "Superduperloop")

How does this improve our WCRTs? Let's again consider the worst-case scenario: a task has become ready to run *juuuust* after the scheduler evaluates its `if` expression as `false`. What's the worst thing that could happen then?

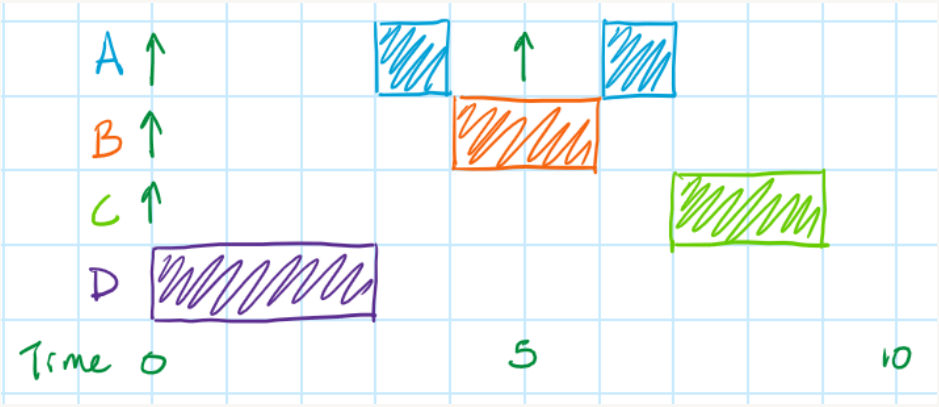
1. First, a lower priority task runs instead. Only it's not any lower priority task, it the *longest* lower priority task that we have in our system. Once it's done, the scheduler starts back at the top of the loop.

2. Next, *every* higher priority task runs, with the scheduler starting back at the top of the loop after each one. Notice that this means that some higher priority tasks can run *multiple times* before our task ever gets to run if, by all of the higher priority tasks running, some of the highest priority tasks are released multiple times.
3. Finally, once no more higher priority tasks are ready to run, the task in question gets to run.

We can express this WCRT in equation (1). Since the higher priority tasks have the ability of running multiple times, we need to use the ceiling function like we did when we evaluated the schedulability of the RTOS in equation (3) in Part 1.

WCRTj=max(WCETi∈i>j)+∑i=0j−1(⌈WCRTj−WCETjPi⌉·WCETi)+WCETj(1)

The term $WCRT_j$ - $WCET_j$ in the summation represents the total amount of time that a higher priority task could potentially run before Task j runs. It might help to represent this with another sequence diagram; we'll assume that we're trying to calculate the response time for Task C.



In this situation, Task A and B get released once each during the time it takes the longest lower priority task (D) to run. *Additionally*, Task A gets released one more time before Task B is finished, so Task A gets to run an additional time. Finally, Task C gets to run.

When we're trying to figure out the maximum amount of time in which the higher priority tasks might get to run, we have to include everything from the start of our WCRT analysis up to the start of Task C. If the cumulative total of all of the blocks is "$WCRT_j$", then that amount of time can be expressed as "$WCRT_j$ - $WCET_j$", since it's the total time minus the last block ($WCET_j$) which is what we have in equation (1).

This is a *significant* improvement over the WCRT from the simple superloop. The response time for the highest priority task is potentially only as long as the execution time of the longest lower priority task, which seems nearly as good as the response time for an RTOS (see equation 3 from Part 1). In fact, it might actually be *better* than the WCRT for an RTOS because once our task starts running we don't have to worry about higher priority tasks preempting our task! The improvement from a simple superloop is so great that I'll call this scheduler the **"Superduperloop"**, though you might see it described in literature simply as a **fixed-priority, non-preemptive scheduler**.

Assigning Task Priorities

Just like for the RTOS, assigning task priorities becomes a critical next step to ensuring that our system is schedulable. Unfortunately, this process can be a bit more complicated than simply using DMS (deadline monotonic scheduling). There are three ways to determine an optimal assignment of priority numbers for our "superloop with task priorities" scheduler that each progress from simple (and only applicable in certain situations) to complex (and applicable in all situations).

Case #1: Tasks with longer deadlines universally have longer WCETs

This case is the most restrictive but it has the easiest priority assignment algorithm. It only works if, after you have sorted your task in order of *increasing* deadline it is *universally true* that your tasks are also sorted in order of *increasing* WCET. The task set below is an example of one such task set that meets this criteria.

Task	Execution time (us)	Period (us)	Deadline (us)
A	1	5	5
B	2	10	10
C	4	12	12

If this condition is met then you can use the DMS algorithm to assign task priorities: the tasks with the shortest deadlines have the highest priorities.

In contrast, the task set below does *not* meet this criteria. Notice how Tasks B and C have longer deadlines but shorter WCETs than Task A.

Task	Execution time (us)	Period (us)	Deadline (us)
A	**3**	5	5
B	**2**	10	6
C	**1**	10	7

Nothing is stopping us from checking if this system is schedulable using DMS, even if the criteria above isn't met, it just means that if the task set *isn't* schedulable when priorities are assigned this way then it *doesn't necessarily* mean that the task set isn't schedulable at all.

If the condition above doesn't hold for a task set (as in the last example), I'd recommend trying to make it work before you give up. Can you run certain tasks slightly faster or relax their deadlines just enough to massage your task set into matching the requirement above (more on this in the section Reducing the WCET of Your Tasks)? Doing so will definitely make assigning priorities much simpler! If you can't, you'll need to use one of the two following procedures.

Case #2a: The General Case, Exhaustive search

For all other cases it's feasible (though inefficient) to just exhaustively try all possible combinations of priority assignments to see which one (if any) work for your task set. You'd probably write a program to help you compute that that would look something like the pseudocode below.

```
FOR EACH combination of priority assignments
    schedulable = TRUE
    FOR EACH task
        IF WCRT of task > Deadline of task    // i.e. task is not schedulable
            schedulable = FALSE
            Break out of inner FOR EACH loop  // i.e. try a new priority
assignment
    IF schedulable
        Break out of outer FOR EACH loop      // If this line of code is reached,
then
                                              // we found a priority assignment
that
                                              // allows every task to be
scheduled!
```

This algorithm has a complexity of $O(n!)$, though, which means it might work fine for task sets with a small number of tasks but it gets less and less feasible if the number of tasks we're trying to schedule gets large. If we can't afford to spend that much time searching for a priority scheme that works (or we simply don't want to be that inefficient), then we'll resort to our last procedure.

Case #2b: The General Case, Audsley's Algorithm

In 1991, Neil Cameron Audsley published a technical report that reduced the problem of searching for this workable set of priority assignments from $O(n!)$ to merely $O(n^2)$. That's a significant improvement! It relied on two key deductions:

1. For a set of n tasks, any workable priority scheme will have one task, and only one task, at each priority level. It doesn't make sense to say that a system is schedulable "as long as Tasks B and C run at the same priority level" since that amounts to saying "this only works if these two tasks run at the same time".
2. To determine if a given task is schedulable at a given priority level, we only need to consider the set of tasks that are either (1) at a higher priority than our task or (2) at a lower priority than our task. The exact priority of those higher or lower priority tasks is of no concern, only whether they exist at a higher- or lower-priority than the task we're considering at the moment. (You can see this in our equation for WCRT above. There's *no part* of that equation that changes the WCRT for Task C if Task B actually has a higher priority than Task A; we'd get the same value for $WCRT_C$ regardless.)

Here's how Audsley's Algorithm works:

1. Start at the lowest priority level for a given set of tasks and see which (if any) tasks can be scheduled assuming *every other unassigned task exists at a higher priority level*. If none of your tasks can be scheduled at the lowest priority level, you're done! Because your system isn't schedulable. :(
2. As long as one task can be scheduled at the lowest priority level, assign it that priority. (If more than one could possibly work at that lowest priority level it doesn't really matter which we pick because we'll still find a workable solution if any such solution exists; this will be easier to grasp once I show you some graphics, below.)
3. Go back to step 1 and repeat with the next higher priority level.

To help you understand this, think about a matrix with our tasks listed across the top and our priority levels listed down the left side, like below.

Priority \ Task	A	B	C	D	E
0					
1					
2					
3					
4					

Based just on which tasks are higher or lower in priority for a given task at a given priority level, we can determine, for each box in our matrix, whether that task will be schedulable. Since tasks with the highest priority will invariably have the smallest WCRT, we can expect the matrix (once it were completed) to look like it does below, with all tasks being schedulable at the highest priority level and then, somewhere down the list of priorities, that task *not* being schedulable anymore.

Priority \ Task	A	B	C	D	E
0					
1					
2					
3					
4					

It's pretty clear from this table that Task E has to have the highest priority, otherwise the system doesn't work. Similarly, next come B and C and then tasks A and D can occupy either of the

lowest two priority levels. (This also shows you why it doesn't matter that we assign the lowest priority to the first task we find that can be schedulable at that lowest priority level; whether we'd found Task A first or Task D, the other one would have been found on the next iteration of the algorithm and we'd have found essentially the same solution.)

Consider a different set of tasks with a table that looks like it does below.

Priority \ Task	A	B	C	D	E
0					
1					
2					
3					
4					

This set of tasks clearly *doesn't* work, exactly because Tasks B and C are only schedulable if they're run at the same priority level (i.e. at the exact same time).

Audley's algorithm works exactly like that: start in the box at the bottom left (lowest priority, first task in the list of tasks) and work your way along the bottom row, stopping when you find a task that's schedulable. Assign it that priority level and then move up one row. If you make your way to the top, you've found a priority scheme that works! If no tasks are schedulable at a given priority level, though (like in that last example), then there isn't *any* priority scheme that will work for your task set. The pseudocode for this algorithm might look like that below.

```
FOR EACH priority level
    schedulable = FALSE
    FOR EACH unassigned task
        IF WCRT of the task < Deadline of the task    // i.e. task is schedulable
at this priority level
            Task priority level becomes the current priority level
            schedulable = TRUE
            Break out of the inner FOR EACH loop
    IF NOT schedulable
        Break out of the outer FOR EACH loop         // If this line of code is
reached, then NO tasks
                                                     // were schedulable at the
given priority level
                                                     // and the task set ISN'T
schedulable.
// IF we get here, though, and schedulable equals TRUE, then the algorithm was
able to assign a task
// at each priority level and the system IS schedulable!
```

If the result of this algorithm is that your task set isn't schedulable, don't give up yet! See if you're able to relax your constraints a little (by optimizing tasks to run faster or relaxing their deadlines or periods) to help make your system schedulable (more on this in the section Reducing the WCET of Your Tasks). Additionally, there's some evidence to suggest that restricting your task periods to be even multiples of each smaller period (like was discussed in the last article) can help make your system more schedulable. If neither solution is feasible or helps, you'll need to consider using a preemptive RTOS, a dynamic priority scheme (more on this in a future article), or even adding another processor to make your system work.

Change #2: Adding Interrupts

For our next change, we'll actually allow interrupts for the first time. Yay!

```
volatile int g_systick = 0;
void timerISR(void)
{
    g_systick++;
}
void uartISR(void) { ... }
void adcISR(void) { ... }
void main(void)
{
    while(1)
    {
        if     ( /* A is ready */ ) { /* Run Task A */ };
        else if( /* B is ready */ ) { /* Run Task B */ };
        else if( /* C is ready */ ) { /* Run Task C */ };
    }
}
```

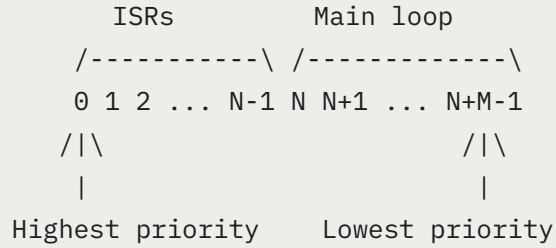
The effect of this change almost goes without saying: our system can now respond extremely quickly to events that would otherwise only get processed once the associated task could run. This is an enormous relief to any system we're designing. Think of the original example I presented in Part 1, in which a system was responding to serial commands at 200k characters/sec and also updating a PD controller: in that example, both the `readSerial` task *and* the `PD` task needed to finish executing *in less time* than it might take a new serial character to arrive, otherwise the system wasn't schedulable. And the `readSerial` task needed to include any time spent actually executing a command once it had been received! Now we can put the short and time-critical portion of `readSerial` (that of receiving a character and placing it into a buffer) into it's own interrupt, with the `readSerial` task only needing to execute as often as we might expect full commands to arrive.

This system isn't without it's downsides, though. Primarily, the improved WCRT that our ISRs enjoy is the result of **hardware preemption** and I made a big point in Part 1 about how fraught

with errors that can be. You'll need to remember the problems that race conditions pose when you write your ISRs, keeping them short and having as few points of interaction with the rest of the system as possible. Ideally, ISRs don't do much more than move data to/from a buffer and/or set a flag to indicate to the main code that they've run. Additionally, by significantly improving the WCRT of certain tasks (the ISRs), we've inadvertently worsened the WCRTs of the lower priority tasks (not unlike an RTOS); more on this in the very next section.

Schedulability of the "Superduperloop" with Interrupts

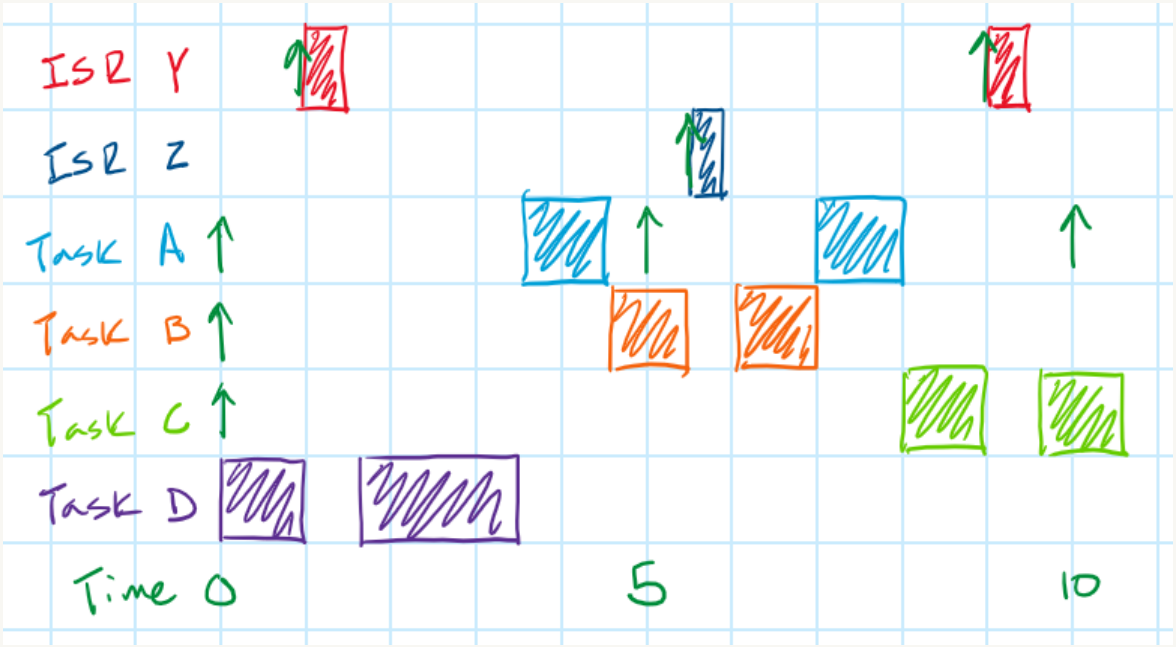
The WCRT of a task in this system is essentially the same as before, with the exception that *any* ISR could run at *any* point throughout a task's WCRT. This means that we'll need to include an additional term with the ceiling function to account for all of that execution time. Also, since we can think of ISRs as being tasks with even higher priorities than the main loop tasks, we're going to re-number things. For a given system, 0 is still the highest priority task, but this now corresponds to an ISR. If the system has N ISRs, then these all have task numbers from 0 to N-1. If this same system has M main loop tasks, then these all have task numbers from N to N+M-1.



The equation below for the WCRT of a task in this system is the same as equation 1 with the addition of the third term, reflecting the total execution time of all ISRs that could run before Task j gets to finish.

WCRTj=max(WCETi∈i>j)+∑i=Nj−1(⌈WCRTj−WCETjPi⌉·WCETi)+∑i=0N−1(⌈WCRTjPi⌉·WCETi) +WCETj(2)

If we modified our example above to include an ISR, it might look like the sequence diagram below.



But that's just the WCRT for each of the *main loop* tasks in our system. Let's now think about the WCRT for the *ISR* tasks in our system.

Thankfully, the WCRTs for the ISRs will look very similar to what we've already seen. The only catch is that we'll have to consider whether our processor will allow ISRs to be preempted or not. If they can be, then their WCRTs looks a whole heck of a lot like the WCRT for an RTOS (equation 3 from Part 1). If they can't, their WCRTs look more like the WCRT for our superloop with priorities (equation 1, above), since ISRs will have to wait for the current ISR to finish before being run in priority order.

ISRsarepreemptible:WCRTj,j<N=∑i=0j−1(⌈WCRTjPi⌉·WCETi)+WCETj(3)
ISRsarenon−preemptible:WCRTj,j<N=max(WCETi,i<N)+∑i=0j−1(⌈WCRTj−WCETjPi⌉·WCETi)
+WCETj(4)

Reducing the WCET of Your Tasks

Since it's the longest running low-priority task that is currently limiting our WCRTs the most, let's consider how we might reduce it's execution time. In fact, the tips below work equally well for reducing the WCET of any task in your system, which will help make it easier to schedule.

- Run the CPU at a faster clock speed.
- Move tasks with shorter deadlines to dedicated co-processors.
- Move work done by a task onto a hardware peripheral.
- Look for speed optimizations in your code or toolchain such as loop unrolling, compiling at a higher optimization level, etc.
- **Break up the task into states.**

It's to the last item that we direct our attention now, since it's the only one that has an effect on how our "Superduperloop" scheduler is constructed and how we calculate the WCRTs of our tasks (since it doesn't affect the overall WCET for a task, just how long it blocks before exiting).

Change #3: Turning Tasks into Finite State Machines

For some tasks, removing all blocking code is fairly non-trivial. What if your task is waiting for a response from a sensor or a remote server? Those portions of code might take a lengthy time! Consider the example below, modeling a task that reads a value from a sensor on a SPI bus, computes a running average, transmits the updated running average somewhere (e.g. over WiFi, over UART, etc) and then waits for an acknowledgement from that server:

```
void TaskA(void)
{
    if( lockSPI() )
    {
        readSensor1();    // Sensor 1 is on the SPI bus
        unlockSPI();
        computeRunningAverage();
        txNewRunningAvg();
        while( !ackReceived() );
    }
}
```

A task like this is ideal for converting into a finite state machine (FSM) since we can put transition points at the places where the task is just waiting for something to happen.

```
void TaskA(void)
{
    static enum { WAITING_FOR_SPI, WAITING_FOR_ACK } state = WAITING_FOR_SPI;
    switch( state )
    {
        case WAITING_FOR_SPI:
            if( lockSPI() )
            {
                readSensor1();    // Sensor 1 is on the SPI bus
                unlockSPI();
                computeRunningAverage();
                txNewRunningAvg();
                state = WAITING_FOR_ACK;
            }
            break;
        case WAITING_FOR_ACK:
            if( ackReceived() ) state = WAITING_FOR_SPI;
            break;
    }
}
```

As in the first code example, our FSM's default state checks if it can acquire a lock on the SPI bus and, if it can, it proceeds to read the sensor and transmit the new running average over a

different network (such as WiFi or something); otherwise it does nothing. Instead of blocking and waiting for an acknowledgement from the server, though, our task merely transitions to a new state, `WAITING_FOR_ACK`. It remains here until the acknowledgement is received, at which point the task begins anew in the `WAITING_FOR_SPI` state. Notice that moving through the FSM (i.e. completing the task) now requires executing the task at least twice (and quite possible more, depending on how long it takes to acquire the lock on the SPI bus or receive the acknowledgement from the server).

Additionally, we can use this technique to break up long chunks of code. Consider the function below, that does some complex math on a long array of numbers.

```
void TaskA(int * array, size_t len)
{
    for( size_t idx = len; idx > 0; idx-- )
    {
        // Complex math on array[len-idx]
    }
}
```

If we just wanted to halve the execution time of this task, we could alter the task so that it executed in two steps or states.

```
void TaskA(int * array, size_t len)
{
    static enum { FIRST, SECOND } state = FIRST;
    switch( state )
    {
        case FIRST:
            for( size_t idx = len; idx > (len/2); idx-- )
            {
                // Complex math on array[len-idx]
            }
            state = SECOND;
            break;
        case SECOND:
            for( size_t idx = (len/2); idx > 0; idx-- )
            {
                // Complex math on array[len-idx]
            }
            state = FIRST;
            break;
    }
}
```

Notice how the two `for` loops now iterate over only half of the array at a time.

Breaking up your tasks into states like this can help reduce their WCET, since the WCET only needs to account for the longest execution of path of a single state, not the entire task. A shorter WCET, particularly for the longest low priority task, can have a significant positive impact on the WCRT of the other tasks and, overall, the ability for your task set to be schedulable with the "Superduperloop" scheduler.

One last piece remains, though, related to how the tasks are executed. Previously the tasks were only released by system conditions (e.g. `charAvailable()` or `PD_next_run - g_systick >= PD_period_ms` [i.e. the system timer]), but now a task that is constructed as an FSM may need to run *multiple times* in order to fully transition through all of its states, even though the original condition may only be true briefly. To do this, we'll need to keep track of when a task is in the middle of it's FSM (and therefore still ready to run). There are many ways to do this, but I'll show you one version, below, that keeps track of each task's "ready to run" status in an integer called `g_tasks`.

```
enum { A_NUM, B_NUM, C_NUM, NUM_TASKS };
const int A_MASK = (1<<A_NUM);
const int B_MASK = (1<<B_NUM);
const int C_MASK = (1<<C_NUM);
volatile int g_systick = 0, g_tasks = 0;
int TaskA(void);
int TaskB(void);
int TaskC(void);
void timerISR(void)
{
    g_systick++;
    g_tasks |= A_MASK;
}
void uartISR(void)
{
    g_tasks |= B_MASK;
}
void adcISR(void)
{
    g_tasks |= (B_MASK | C_MASK);
}
void main(void)
{
    while(1)
    {
        if     ( g_tasks & A_MASK ) g_tasks = TaskA() ? g_tasks|A_MASK :
g_tasks&(~A_MASK);
        else if( g_tasks & B_MASK ) g_tasks = TaskB() ? g_tasks|B_MASK :
g_tasks&(~B_MASK);
        else if( g_tasks & C_MASK ) g_tasks = TaskC() ? g_tasks|C_MASK :
g_tasks&(~C_MASK);
```

```
    }
}
```

ISRs or other tasks set the appropriate bit in the `g_tasks` variable to indicate that a task has been released. Checking to see if a task is ready to run then becomes as simple as checking if it's bit in `g_tasks` is set. Tasks have each been extracted into their own functions (`TaskA()`, etc) and each function returns a value corresponding to what the value of their flag should be, 1 for "continue running" (i.e. FSM has not yet finished) and 0 for "task is finished" or "task is waiting on another event". In this manner, each task can continue running once its been released by returning a 1 until the FSM is complete (or waiting on another event).

Dealing with Race Conditions

Now that we are modifying variables from both the ISRs and the main loop, though, we have to worry about those dreaded race conditions! Consider the awfulness of the scenario below:

1. The `timerISR` runs, releasing Task A.
2. Task A completes and returns a 0.
3. `g_tasks` is read from memory, in preparation for clearing the zeroth bit. Let's say it looks like 0b001; the main code clears the zeroth bit so 0b000 is the value stored in a register somewhere, ready to be written back to the location in memory where `g_tasks` resides.
4. Right at that *exact* moment, the `uartISR` runs, setting the first bit in `g_tasks`. The ISR reads the original `g_tasks` value from memory (0b001) and then stores 0b011 back to memory.
5. Execution returns to the machine instruction that is updating `g_tasks` after running Task A, only now the first bit gets *cleared* since it wasn't set when the main code read the `g_tasks` variable in step 3!

This means that we have a **critical section** every time `g_tasks` is modified by a piece of code that could be preempted by an ISR (which includes ISRs, if ISRs are preemptible!). The three most straightforward ways to protect a critical section and avoid the awfulness of above are:

1. disable interrupts before the critical section (and re-enable them after),
2. use atomic instructions (either `stdatomic.h` or compiler extensions [such as for GCC]) to ensure that a variable isn't overwritten, or
3. require that a mutex or lock be acquired prior to entering the critical section.

Disabling interrupts is the easiest and simplest but adds to the time that an ISR is blocked, proportional to the number and length of the critical sections. Mutexes are more surgical, allowing ISRs to still run, but are non-trivial to implement and might result in an ISR effectively being ignored (if an ISR interrupts the critical section and attempts to acquire the lock but can't). Atomic instructions avoid any blocking time from disabling interrupts but they do require support from the processor, specifically a set of instructions that will abort any write to a memory location if that memory location was modified after it was previously read. These instructions are referred to generally as "load-linked" and "store-conditional" instructions. Had

these instructions been used above, the write in step #5 would have failed, preserving the value of `g_tasks` that the `uartISR` wrote. The functions in `stdatomic` (available in C11/C++11 and onward) or that are provided as compiler extensions create a small loop where the memory location in question is read from (using the "load-linked" instruction), modified, and written back to until the "store-conditional" part succeeds, as can be seen on Compiler Explorer. (Of note is that the Cortex-M0/M0+ lacks this support, so that leaves you with needing to disable/re-enable interrupts on that processor to protect your critical sections, unless you want to dive into the world of lock-free data structures.)

I'll use the atomic functions in `stdatomic.h` for the remainder of this series to protect the critical sections that we encounter, using either `atomic_fetch_or` or `atomic_fetch_and` to update `g_tasks`.

```
if( g_tasks & A_MASK ) TaskA() ? atomic_fetch_or(&g_tasks, A_MASK) :
atomic_fetch_and(&g_tasks, (~A_MASK));
```

There's actually ***another*** race condition that's been present this whole time which we haven't dealt with yet, too: the reading of `g_systick` or `g_task` by one of the tasks. If the number of bits in these globals is more than the width of our processor (meaning that reading `g_systick` or `g_task` would take more than one memory operation, one to read first the high byte and then another to read the low byte, for example) then we have to contend with the following:

1. A task reads the high byte of `g_systick`. Say `g_systick` is 0x0000FFFF, so 0x0000 is read.
2. The timer ISR runs at that *exact moment*, incrementing `g_systick` to 0x00010000.
3. Execution returns to the task which reads the lower byte of `g_systick` which is now *0x0000*! The task gets the value of 0x00000000, which is very incorrect!

This is a type of race condition called a "torn read". To ensure that any value of a global variable we read is a valid one, your options are the same as above (disable interrupts before and re-enable them after reading the variable or use an atomic function such as `atomic_load` from the `stdatomic` library); you can also use the following snippet of code, which essentially works by reading `g_systick` until two subsequent reads have the same value, assuming that it won't have been updated by any ISR if it holds the same value two times in a row.

```
int systick_val;
do
{
    systick_val = g_systick;
} while ( systick_val != g_systick );
```

Additionally, there's ***still one more*** race condition to consider!! Imagine the following sequence of events, using the task above that read from a sensor and sent out the running average over a different network:

1. The task executes its first state: reading the sensor, sending it out, and changing it's state variable to WAITING_FOR_ACK. It returns a 0 to the main loop to clear its ready flag, indicating that it shouldn't run again until some event causes it to be released (such as an acknowledgement being received).
2. Before the task's ready flag gets cleared in the main loop, the acknowledgement is received, setting the task's ready flag (which is still 1 from the event that caused the task to run just now).
3. The task returns and tells the main loop to clear its ready flag. Now it's waiting for an event that has already occurred!

(Sidenote: Isn't it a pain to have to identify and handle all of the possible race conditions with this *small* chunk of code and only *two* shared variables? Can you imagine trying to get all of this correct with a preemptive RTOS, when *any* task can interrupt *any* lower-priority task in between *any* of its machine instructions, many of which can be *reordered* around each other by either the compiler or the processor itself?!)

Anyway, there are probably multiple solutions to this problem, but the simplest one to me is to clear each task's ready flag before it runs and only allow it to be **set** afterwards, if the task needs to run again.

```
if( g_tasks & A_MASK )
{
    atomic_fetch_and(&g_tasks, (~A_Mask));
    if( TaskA() ) atomic_fetch_or(&g_tasks, A_MASK);
}
```

Any event that releases the task up to the point where `atomic_fetch_and` successfully writes to `g_tasks` should get handled when the task runs. Any event that releases the task *after* the ready flag is cleared may or may not be handled when the task runs, but since the task is only allowed to *set* its ready flag after its completes (not clear it), then even if that event isn't handled on the current invokation of the task, the task will run at least once more before the system goes to sleep or handles a lower-priority task, ensuring that the event will eventually be handled. (There isn't a way for the scheduler to know if an event arrives out of order [e.g. if an acknowledgement arrived before the sensor data was sent out!], so each task will have to handle that synchronization on their own.)

So the "Superduperloop" should *really* look like this:

```
enum { A_NUM, B_NUM, C_NUM, NUM_TASKS };
const int A_MASK = (1<<A_NUM);
const int B_MASK = (1<<B_NUM);
```

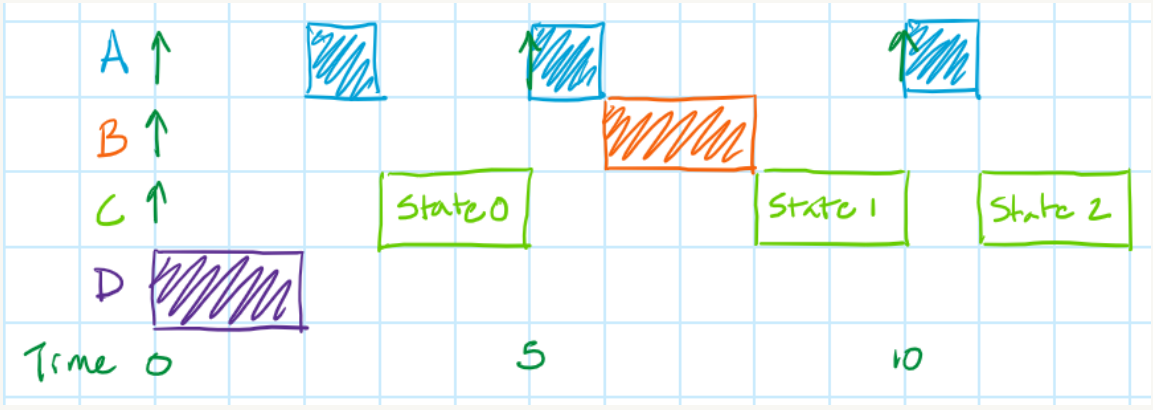
```
const int C_MASK = (1<<C_NUM);
volatile int g_systick = 0, g_tasks = 0;
int TaskA(void);
int TaskB(void);
int TaskC(void);
void timerISR(void)
{
    g_systick++;
    atomic_fetch_or(&g_tasks, A_MASK);  // Assuming ISRs are preemptible.
Otherwise
                                        // "g_tasks |= A_MASK;" is fine.
}
void uartISR(void)
{
    atomic_fetch_or(&g_tasks, B_MASK);  // Assuming ISRs are preemptible
}
void adcISR(void)
{
    atomic_fetch_or(&g_tasks, B_MASK|C_MASK);  // Assuming ISRs are preemptible
}
void main(void)
{
    while(1)
    {
        if( g_tasks & A_MASK )
        {
            atomic_fetch_and(&g_tasks, (~A_Mask));
            if( TaskA() ) atomic_fetch_or(&g_tasks, A_MASK);
        }
        else if( g_tasks & B_MASK )
        {
            atomic_fetch_and(&g_tasks, (~B_Mask));
            if( TaskB() ) atomic_fetch_or(&g_tasks, B_MASK);
        }
        else if( g_tasks & C_MASK )
        {
            atomic_fetch_and(&g_tasks, (~C_Mask));
            if( TaskC() ) atomic_fetch_or(&g_tasks, C_MASK);
        }
    }
}
```

Tedious, I know, but it's the price we pay for preemption. If you can make your system work without ISRs, then you can avoid all but the last of these critical sections altogether!

Schedulability of the "Superduperloop" with FSMs

Okay, NOW what's the worst thing that could happen to block a task from running? The worst-case scenario is the same as for the superloop with task priorities and interrupts, except that, now, higher-priority tasks have the chance of running all the way until the start of the last FSM state.

For the "superloop with task priorities and interrupts", no higher priority task was allowed to run after the task in question started executing (since the superloop is a cooperative scheduler, not a preemptive scheduler). You can see this from 7 to 9 usec in the sequence diagram, when Task C was allowed to run uninterrupted. For the updated "Superduperloop", since the task in question may return in between each state higher priority tasks are allowed to run all the way until the start of the *final state* of the task in question (of course, this is exactly what we wanted to happen!). This might look like the sequence diagram below. Notice now how higher priority tasks are allowed to run all the way until 11 usec, at the start of the final state for Task C (assuming that Task C has a total of 3 states).



We can represent this mathematically by changing the numerator in the second term of equation 2 (which described the total length of time that the higher-priority tasks could run) from $WCRT_j - WCET_j$ to $WCRT_j - WCET_{j,Final\ state}$, which is the execution time of the last state in Task j's FSM. (If Task j isn't an FSM, then this term devolves back into $WCRT_j - WCET_j$.)

$$WCRTj=\max(WCETi\in i\>j)+\sum i=Nj-1(\lceil WCRTj-WCETj,FinalstatePi\rceil\cdot WCETi) +\sum i=0N-1(\lceil WCRTjPi\rceil\cdot WCETi)+WCETj(5)$$

Cool Projects to Check Out: Protothreads and "Coroutines in C"

If you're getting tired of writing long FSMs and don't feel squeamish about strange uses of the C language, then you might be interested in checking out either the Protothreads project or the article "Coroutines in C", which both make writing FSMs easier by wrapping all of the `switch...case` syntax above into several function-like macros. For example, here's what the task above (that was reading a sensor and sending out an updated value for the running average) would look like in Protothreads:

```
PT_THREAD(TaskA(struct pt *pt))
{
    PT_BEGIN(pt);
    PT_WAIT_UNTIL( pt, lockSPI() );
    readSensor1();        // Sensor 1 is on the SPI bus
    unlockSPI();
    txNewRunningAvg();    // Compute running average
    PT_WAIT_UNTIL( pt, /* Ack received */ );
    PT_RESTART(pt);
    PT_END(pt);
}
```

Clean, isn't it? And here's what it would look like after the C preprocessor has had a chance to expand all of those macros:

```
char TaskA(struct pt *pt)
{
    char PT_YIELD_FLAG = 1;
    switch((pt)->lc)
    {
        case 0:
            do
            {
                (pt)->lc = 7;
        case 7:
                if(!(lockSPI())) { return 0; }
            } while(0);
            readSensor1();
            unlockSPI();
            txNewRunningAvg();
            do
            {
                (pt)->lc = 11;
        case 11:
                if(!(ack())) { return 0; }
            } while(0);
            do
            {
                (pt)->lc = 0;
                return 0;
            } while(0);
    };
    PT_YIELD_FLAG = 0;
    (pt)->lc = 0;
    return 3;
}
```

The `PT_BEGIN` and `PT_END` macros just form the beginning and end of a `switch...case` statement. The `PT_WAIT_` macros insert new case statements and set the internal state variable (which is `pt->lc`) accordingly so that the next time the task is called it jumps to that same case statement. The `PT_RESTART` macro starts the FSM back at the top, with the line of code just below `PT_BEGIN`. Protothreads is a neat project and even includes support for a basic form of semaphores, so we may look at it again in the future!

Bonus change!: Going to Sleep

Okay, so I've been ducking the question for nearly two full articles now: "What happens when there's no work to be done?? When does my application go to sleep??" Truth be told, I've been waiting until we'd been able to discuss the "Superduperloop", since analyzing the schedulability of any of the earlier superloop variations gets weird if the scheduler has the ability to go to sleep at some point. Thankfully, at this point, going to sleep is a simple change and does not affect the schedulability analysis we've done so far of the "Superduperloop".

Our "Superduperloop" should go to sleep *only* when there's no work to be done. This would occur exactly if none of the `if`/`else if` expressions were true and the `if` statement were to try to execute an `else` block. So that's where we'll put our code to go to sleep!

```
if( g_tasks & A_MASK )      { ... }
else if( g_tasks & B_MASK ) { ... }
else if( g_tasks & C_MASK ) { ... }
else /* Go to sleep here */;
```

The "go to sleep" code needs to be a *touch* more complicated than just inserting a `wfi();` or similar piece of code. The problem we want to avoid is the (admittedly unlikely, but still possible) case when, in between checking the last expression and then going to sleep, an interrupt runs and releases a task. This would result in a task being ready to run but the system going to sleep anyways! To avoid this, we'll use something called a "double-checked lock".

```
if( g_tasks & A_MASK )      { ... }
else if( g_tasks & B_MASK ) { ... }
else if( g_tasks & C_MASK ) { ... }
else
{
    if( !g_tasks )
    {
        disableInterrupts();
        if( !g_tasks ) wfi();
        enableInterrupts();
    }
}
```

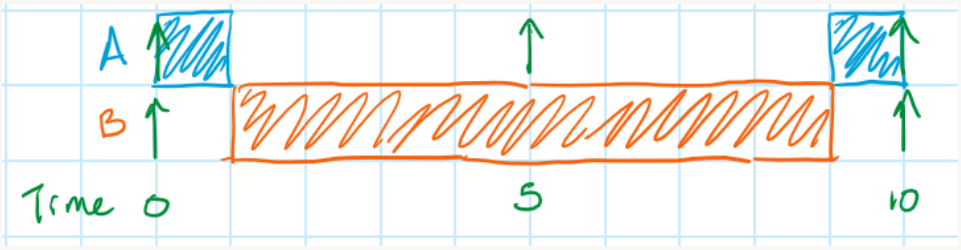
The double-checked lock first checks if any task is ready to run and then, if none are, disables all interrupts, preventing any ISRs from running (but still letting them pend). (Checking the lock the first time isn't strictly required, but it does save us the trouble of disabling and immediately re-enabling interrupts if there's a task that's ready to run.) The `g_tasks` variable is checked a second time (just to make sure that no ISR released a task right before interrupts were disabled) and then, as long as no task is ready to run, the system goes to sleep. The code used to put the processor to sleep should return immediately if there are any pending ISRs (to catch the *just as unlikely* but still possible case that an ISR pended (but didn't run) after interrupts were disable) but otherwise allow the processor to go to sleep. Once asleep, the processor should wake up for any pending ISR, which will get to run immediately once interrupts are re-enabled. Then the scheduler loop will start again from the top.

Alternative Design: The Time-triggered Scheduler

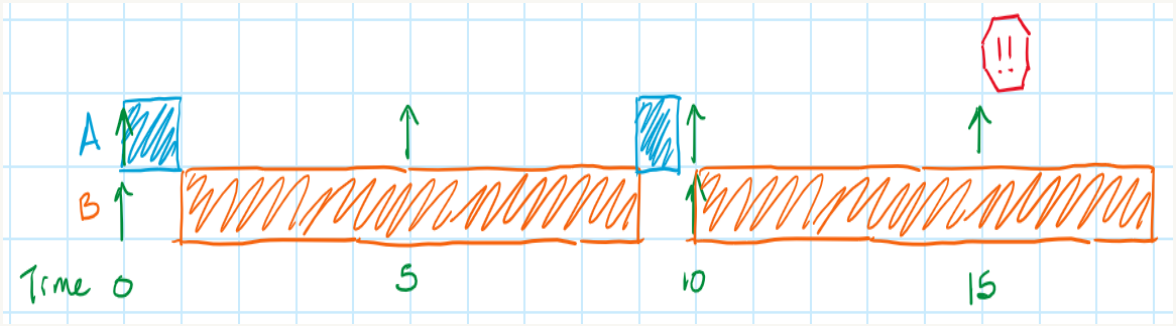
The "Superduperloop" is a very capable scheduler (if I do say so myself!), but even *it* can't schedule certain task sets that seem like they *should* be schedulable. Consider the task set, below, for instance.

Task	Execution time (us)	Period (us)	Deadline (us)
A	1	5	5
B	8	10	10

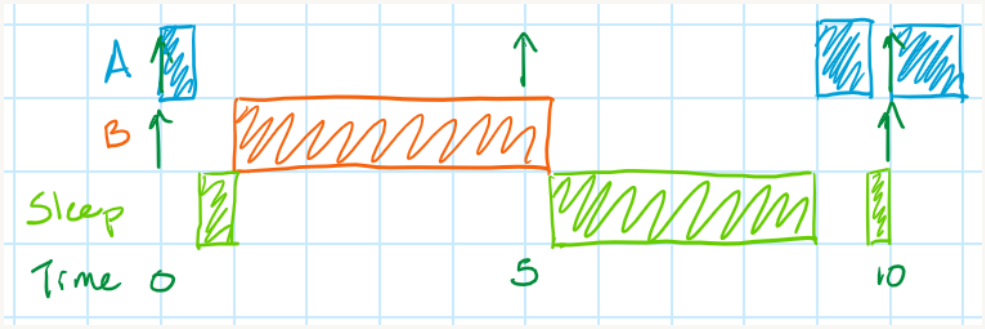
Utilization is the highest it could be, 100%, but shouldn't this still be schedulable? Doesn't the sequence diagram below prove it?



Unfortunately, the answer is "no"! (Or perhaps it's *fortunate* that the answer is "no"? Otherwise our schedulability analyses would be all wrong!) Remember that the WCET for a task is its *worst-case* execution time; many times the task will actually run much faster. Consider the possibility that the second iteration of Task A above took only 0.999... usec to run instead of the full 1 usec. The "Superduperloop" would return to the top of the `for` loop and start looking for the next ready to run task. Suppose that Task A finished early enough that, on this next pass, the timer *hadn't* incremented yet and Task A *wasn't* ready to run. Suppose then that, *immediately* after this, the timer *did* increment and Task B became ready to run. Then *it* would be executed, taking its full 8 usec, causing Task A to miss it's deadline! What a bummer! This result, though, *does* agree with our WCRT analysis from Part 1 of this series.



So what's a designer to do if we have a system that *looks* schedulable but isn't, based on the math? One option is to manually insert idle times that force the scheduler to stick to a schedule that we know works, like "Run Task A for 1 usec (going to sleep if it finishes early), then Task B for 8 usec (again going to sleep if necessary), and lastly Task A again for 1 usec (going to sleep again as necessary)". This is called a **time-triggered** scheduler or a scheduler with **inserted idle times**. Using this type of scheduler, the sequence diagram might look as sparse as below, but the trade-off is that we *know* when each task will run and that the system can be made to be schedulable. Implementation of this type of scheduler is left as an exercise for the reader.



The biggest problem with this scheduler is that you'll need to specify exactly which task needs to run for the length of the **hyperperiod**, which is the least common multiple of all of the task periods. This is the period over which each of the task's releases repeat themselves, so defining which task is running over the entire hyperperiod allows the system to be fully defined. It's *not* particularly feasible to do that if the hyperperiod is large, which is not hard to do even for a small number of tasks if their periods are very unrelated. For example, the hyperperiod of tasks with periods of 5, 11, and 23 usec is 1,265 usec!

"In the blue corner...!"

We might have compared superloops and RTOSes in the last article, but that wasn't a fair fight! Let's give the RTOS a run for it's money by doing a side-by-side with the "Superduperloop".

"Superduperloop"	RTOS

`// One enum value and bitmask per task, e.g.`
`enum { A_NUM, B_NUM, NUM_TASKS };`
`const int A_MASK = (1<<A_NUM);`
`const int B_MASK = (1<<B_NUM);`
	`void main(void)`
	`{`
	`  taskCreate(TaskA, 0);  // Priority 0`

```
// Function prototypes for tasks here, e.g.
int TaskA(void);
int TaskB(void);
volatile int g_systick = 0, g_tasks = 0;
void timerISR(void)
{
    g_systick++;
    // Release a task using, e.g.
    // atomic_fetch_or(&g_tasks, A_MASK);  //
Assuming ISRs are preemptible. Otherwise
                                           //
"g_tasks |= A_MASK;" is fine.
}
void main(void)
{
    while(1)
    {
        // One if/else if block per task,
higher priority tasks located
        // higher up in the chain of if
statements.
        //
        if( g_tasks & A_MASK )
        {
            atomic_fetch_and(&g_tasks,
(~A_Mask));
            if( TaskA() )
atomic_fetch_or(&g_tasks, A_MASK);
        }
        else if( g_tasks & B_MASK )
        {
            atomic_fetch_and(&g_tasks,
(~B_Mask));
            if( TaskB() )
atomic_fetch_or(&g_tasks, B_MASK);
        }
        else
        {
            if( !g_tasks )
            {
                disableInterrupts();
                if( !g_tasks ) wfi();
                enableInterrupts();
            }
        }
    }
```

```
(highest)
    taskCreate(TaskB,
1);  // Priority 1
    taskCreate(TaskC,
2);  // Priority 2

startScheduler();      //
No return
}
void TaskA(void)
{
    while(1)
    {
        // Task A
    }
}
void TaskB(void)
{
    while(1)
    {
        // Task B
    }
}
void TaskC(void)
{
    while(1)
    {
        // Task C
    }
}
void onIdle(void)
{
    wfi();
}
```

`}`	
Tasks: $ \quad WCRT_{j \ge N} = max(WCET_{i \in i>j}) + \sum_{i=N}^{j-1} ({\left \lceil \frac {WCRT_j - WCET_{j,Final state}} {P_i} \right \rceil \cdot WCET_i}) \quad $ $ \quad + \sum_{i=0}^{N-1} ({\left \lceil \frac {WCRT_j} {P_i} \right \rceil \cdot WCET_i}) + WCET_j \quad $	Tasks: $ \quad WCRT_{j \ge N} = \sum_{i=0}^{j-1} ({\left \lceil \frac {WCRT_j} {P_i} \right \rceil \cdot WCET_i}) + WCET_j \quad $
ISRs: see equations 3 and 4	ISRs: see equations 3 and 4
Optimal priority assignments: • DMS • Exhaustive search • Audsley's algorithm	Optimal priority assignments: DMS

(If the chain of `if` statements is hurting your soul at this point, I encourage you to rewrite this little scheduler using a `for` loop that iterates over a list of prioritized tasks instead! It's a great exercise.)

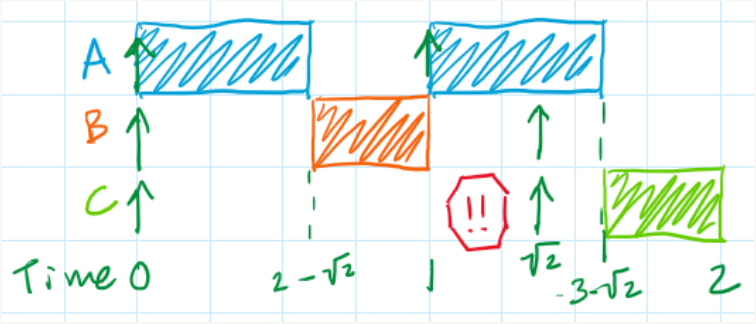
It's fascinating to me that, despite the WCRT equation for the RTOS having fewer terms, there's *nothing* to indicate that it is *always* a smaller value than for the "Superduperloop"! In fact, for some task sets, the "Superduperloop" *works* when the RTOS *doesn't*! Let's take a look at that situation, shall we?

The Curious Case of the "Superduperloop" Besting an RTOS!!

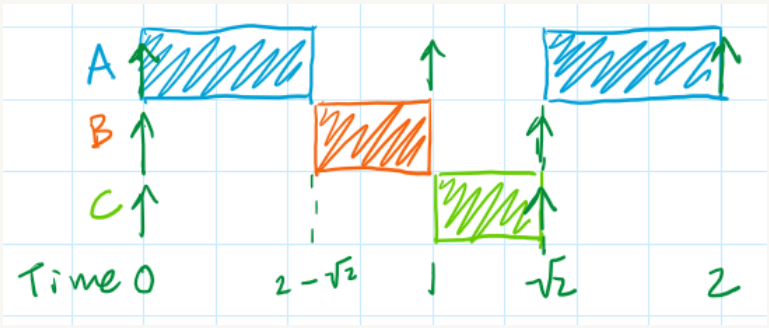
Sometimes the fact that a scheduler *can't* preempt a lower-priority task is actually an *advantage*, instead of a liability. Consider the task set below.

Task	Priority	Execution time (us)	Period (us)	Deadline (us)	WCRT (RTOS)	WCRT (Superduperloop)
A	0	$2 - \sqrt 2$	1	1	$2 - \sqrt 2$	1
B	1	$\sqrt 2 - 1$	10	$\sqrt 2$	1	$\sqrt 2$
C	2	$\sqrt 2 - 1$	10	$\sqrt 2$	**2!!**	$\sqrt 2$

Using an RTOS, this task set ***isn't schedulable***! Task A preempts Task C at 1 usec owing to its higher priority, causing Task C to miss its deadline. Bummer!



However, it **is** schedulable using the "Superduperloop"!! Because Task C *can't* be preempted, it finishes before its deadline, leaving Task A enough time to also finish before *it's* deadline. Success!



This isn't a fluke, either; it's the result of the two WCRT equations above being different enough that *neither* is universally larger or smaller than the other. Some task sets will be schedulable using the "Superduperloop" but not the RTOS, some will be schedulable using the RTOS but not the "Superduperloop", and some task sets will be schedulable with either or neither. You'll simply need to check to see which, if either, of the two schedulers will work for your task set. Superloops, it turns out, aren't something you outgrow when you leave school and move on to more advanced embedded system projects; they might very well be the only scheduler that allows your system to work!

Cool Project to Check Out: RIOS

RIOS ("Riverside-Irvine Operating System") is a project to come out of the University of California, Riverside, and University of California, Irvine, aimed at building the smallest, capable cooperative and preemptive schedulers possible for the purposes of teaching students how operating systems work (not unlike what we're trying to do here!). The cooperative version of their scheduler (below) actually looks a lot like the "Superduperloop"!

Figure 5: Sample program using non-preemptive RIOS with state machine tasks.

```
unsigned char processingRdyTasks = 0;
void TimerISR() {
  unsigned char i;
  if (processingRdyTasks) {
    printf("Timer ticked before task processing done.\n");
  }
  else { // Heart of the scheduler code
    processingRdyTasks = 1;
    for (i=0; i < tasksNum; ++i) {
      if (tasks[i].elapsedTime >= tasks[i].period) { // Ready
        tasks[i].state = tasks[i].TickFct(tasks[i].state);
        tasks[i].elapsedTime = 0;
      }
      tasks[i].elapsedTime += tasksPeriodGCD;
    }
    processingRdyTasks = 0;
  }
}
```

Update state

RIOS scheduler

The biggest exception is probably that tasks in RIOS are *all* periodic, while the "Superduperloop" is designed to handle event-driven tasks also. They also handle integer rollover slightly differently than we did in the first article, by keeping track of each tasks' elapsed time and resetting that value to 0 every time the task gets run.

The team also managed to write a preemptive RTOS in about two dozen lines of code, which is definitely worth a look. You can read more about RIOS in their original paper or at a university-hosted website.

What's Missing from the "Superduperloop"?

Our cooperative scheduler has met the original goal of having comparable WCRTs to that of an RTOS while (almost) completely avoiding the possibility of race conditions. The *last* edge that the RTOS has over us is its OS primitives: mutexes, semaphores, message queues, stream buffers, event flags, and thread flags (depending on which RTOS you're using). When tasks *do* interact, these tools are extremely useful for synchronizing and coordinating those interactions. In particular, our current system of task triggering is pretty limited. It's kind of like each task has a thread flag that's only 1 bit wide: the task can block on just this one trigger and when it finally does run it has to check which, of possibly multiple, events was the one that triggered it.

It's to this problem that we'll turn our attention to next. Adding these synchronization and triggering mechanisms to the "Superduperloop" is not all that hard so we'll take a look at how to do that in the next article. Additionally, we'd be remiss not to consider what other projects (commercial or open-source) are available to us that have already solved these problems. By the end of that third article, I hope you have, not one, but several implementation options for the

"Superduperloop" (or similar) that are fully competitive with whatever RTOS you are currently using or were planning to use in the future.

Summary

In this article, we improved on the simple superloop by adding task priorities, interrupts, and FSMs to create a thing I'm calling the **Superduperloop** (a.k.a. the **fixed-priority, non-preemptive scheduler**). The "Superduperloop" is a cooperative scheduler that has a WCRT for most tasks that comes close to the WCRT of a preemptive RTOS (or even exceeds it!), making it highly suitable for scheduling many embedded applications. It's simple enough to write in a few dozen lines of code and it eliminates the possibility of race conditions (ISRs notwithstanding), making it

- simple to understand,
- robust,
- incredibly small (RAM- and Flash-wise),
- and eminently portable.

It *doesn't*, unfortunately, have many useful OS primitives such as mutexes, semaphores, message queues, stream buffers, event flags, or thread flags. Most of these are not difficult to add to the "Superduperloop" but also there are some notable commercial and open-source projects that have tackled that problem already. Stay tuned, my friends, when we turn the "Superduperloop" into a fully-fledged cooperative OS in the next two articles in this series!

References

- Multi-Rate Main Loop Tasking ("Better Embedded System SW" blog) by Philip Koopman
- Better Embedded System Software by Philip Koopman
- Embedded Systems Fundamentals with ARM Cortex-M based Microcontrollers, Chapter 3, by Alexander Dean
- Davis, R. I., Cucu-Grosjean, L., Bertogna, M., & Burns, A. (2016). A review of priority assignment in real-time systems. *Journal of Systems Architecture*, *65*, 64–82. https://doi.org/10.1016/j.sysarc.2016.04.002
- A lot of my notes come from taking Professor Alex Dean's classes at NC State: ECE560 (Embedded Systems Architectures) and ECE561 (Embedded Systems Design/"Optimization"). His course notes reference, among others, the following articles and books:
    - George, L., Rivierre, N., & Spuri, M. (1996). *Technical Report RR-2966: Preemptive and Non-preemptive Real-time Uniprocessor Scheduling.* INRIA.
    - Sha, L., Abdelzhaer, T., Arzen, K.-E., Cervin, A., Baker, T., Burns, A., et al. (2004, November-December). Real Time Scheduling Theory: A Historical Perspective. *Real Time Systems*, 28(2-3), 101-155.

- Davis, R. I., Thekkilakattil, A., Gettings, O., Dobrin, R., Punnekkat, S., & Chen, J.-J. (2017). Exact speedup factors and sub-optimality for non-preemptive scheduling. *Real-Time Systems*, *54*(1), 208–246. https://doi.org/10.1007/s11241-017-9294-3
- Real-time Systems and Programming Languages by Alan Burns and Andy Wellings
- A Practitioner's Guide Handbook for Real-time Analysis by Klein, Ralya, Pollak, Obenza, and Harbour.